

Artificial Neural Networks (ANN)

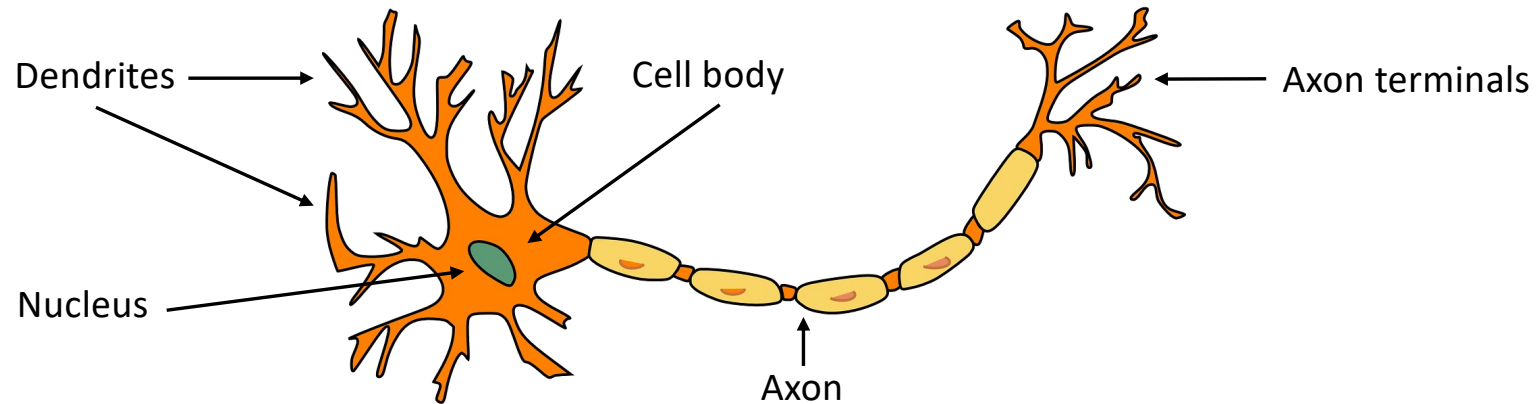
Jun Hong

Part 1 – Artificial Neural Networks

- There are two parts related to deep neural networks
- Part 1: Artificial neural networks (ANN)
 - We'll recap/review ANN which provides the basics of neural networks and the basis for Part 2 on deep neural networks
- Part 2: Convolutional neural networks (CNN)
 - The basic form and the basics of deep neural networks

Biological neuron

A neuron is the most basic unit of our nerve system



- It receives and *aggregates* sensory/electrical signal (*input*) from neighbour neurons
- It sends electrical messages/signals (*output*) to other neurons when activated
- Biological neurons can get *activated*, depending on their input signals

Biological neuron

A human perceives and reason about the world based on the *interconnections/networks of neurons* in our brain



Biological neuron

A human perceives and reason about the world based on the interconnections/networks of neurons in our brain



Biological neuron

A human perceives and reason about the world based on the interconnections/networks of neurons in our brain



Biological neuron

A human perceives and reason about the world based on the interconnections of neurons in our brain

Input



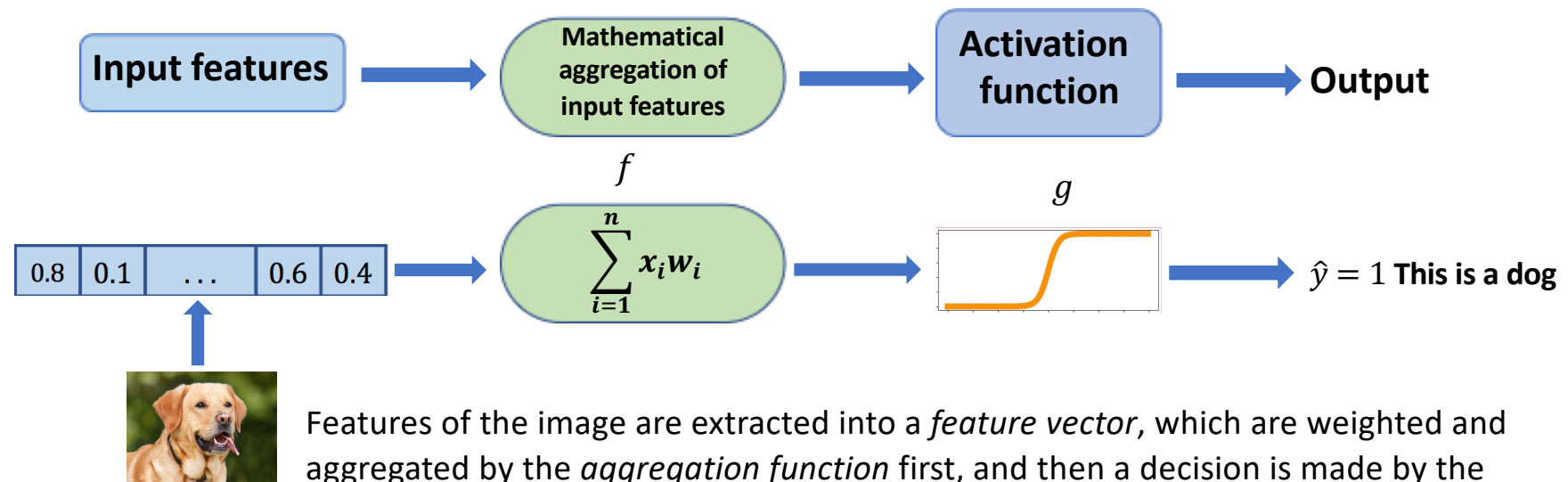
Output

This is a dog

Artificial Neuron

Logistic regression is a special case in which Sigmoid is used as the activation function.

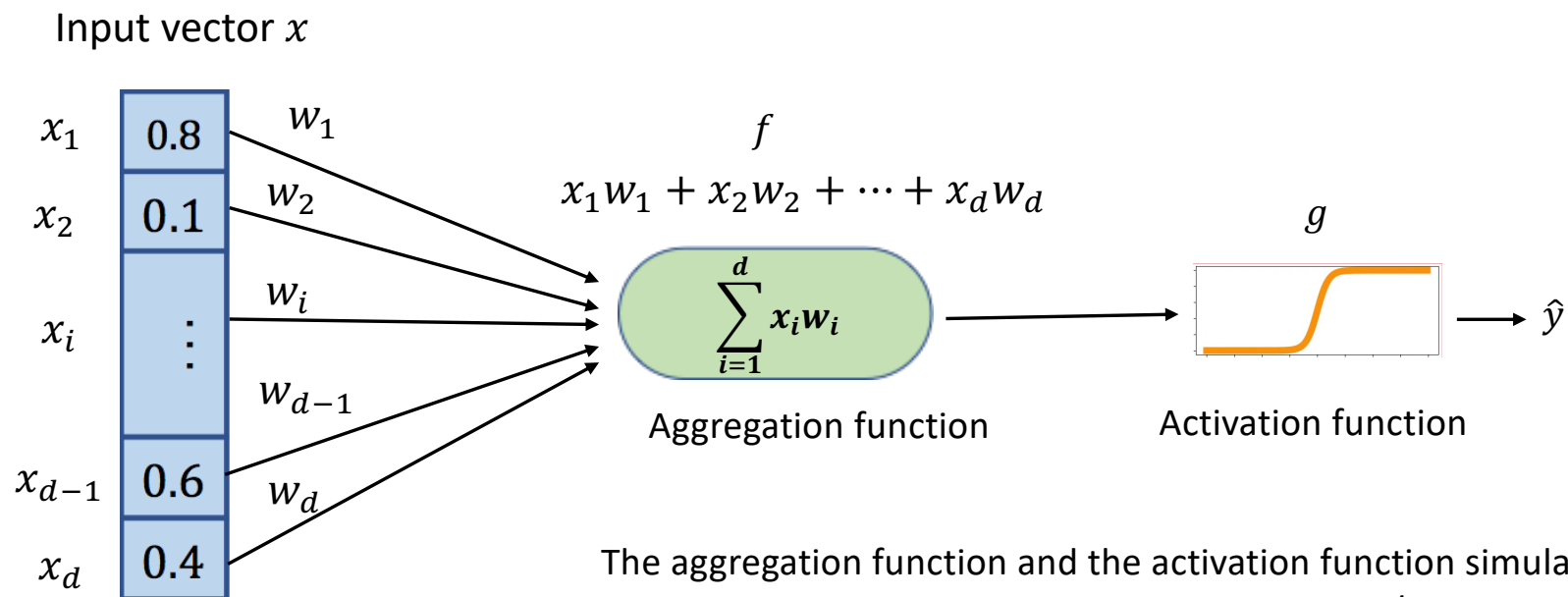
- An artificial neuron is the most basic unit of an Artificial Neural network (ANN)
- It is inspired by the biological neuron in our brains



Features of the image are extracted into a *feature vector*, which are weighted and aggregated by the *aggregation function* first, and then a decision is made by the *activation function* based on the value of the weighted summation.

Artificial Neuron

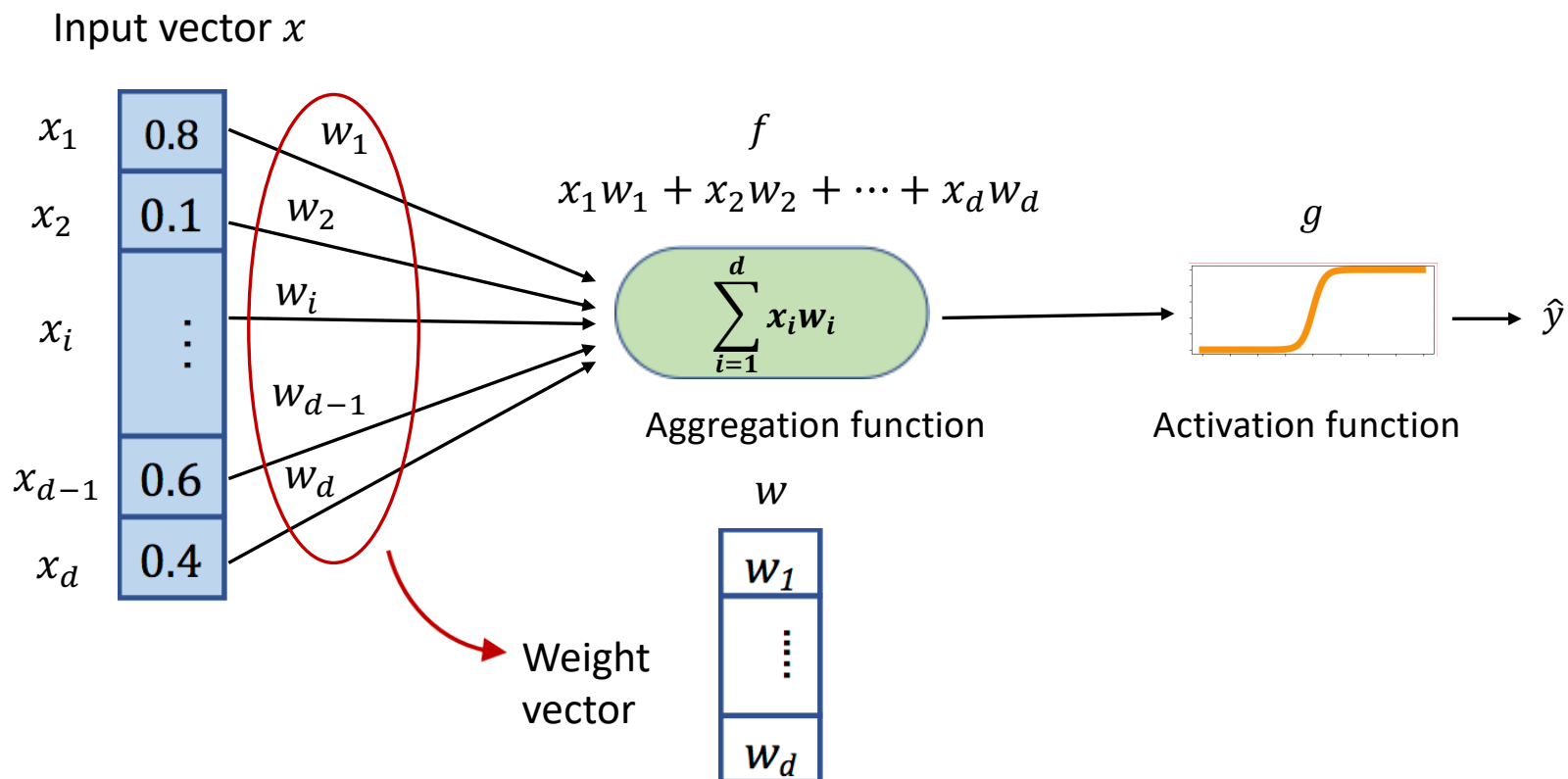
Logistic regression is a special case in which Sigmoid is used as the activation function.



The aggregation function and the activation function simulate how a neuron receives inputs, aggregates and activates/produces the output

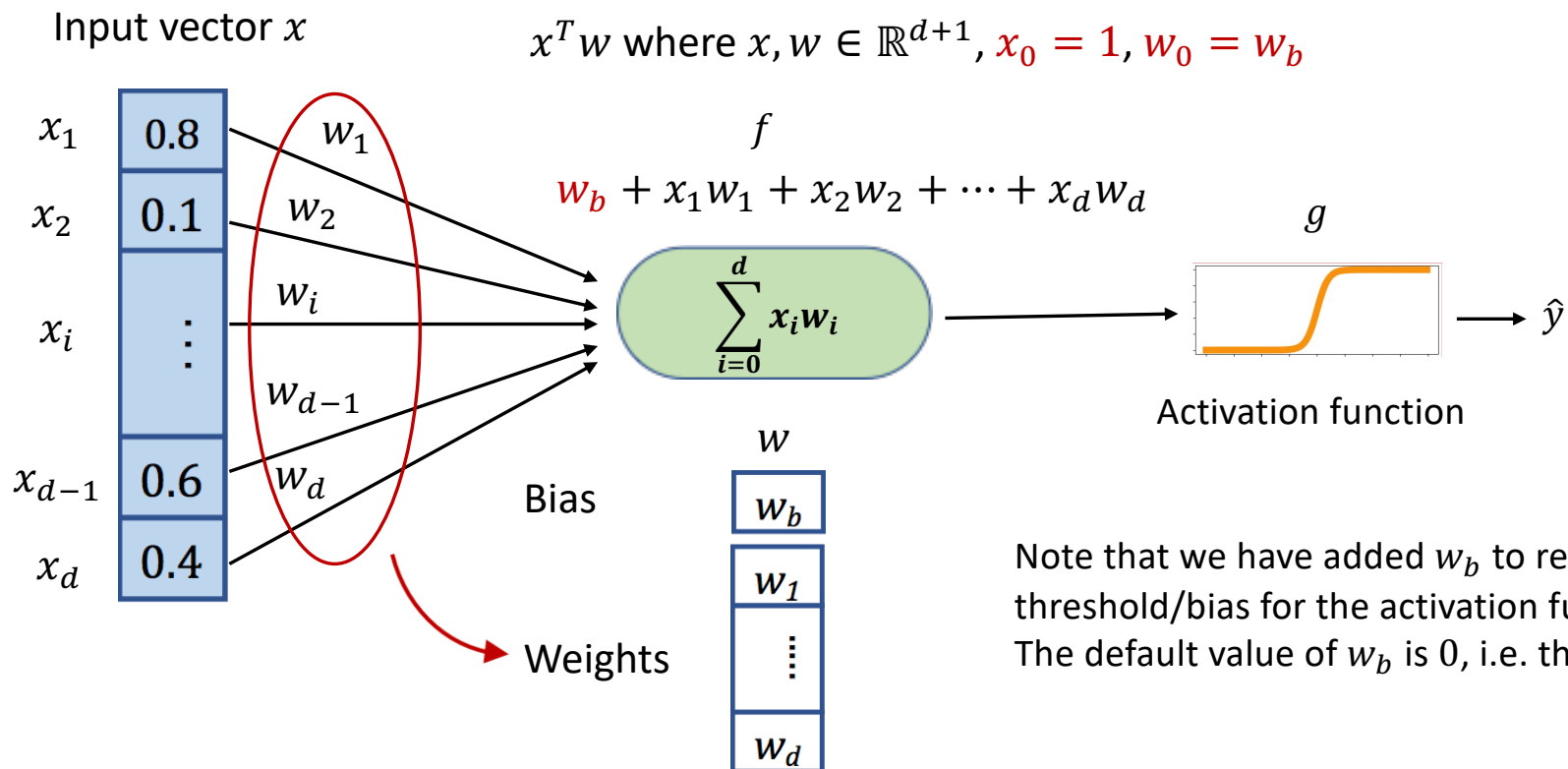
The values of x_i are in the range of $[0, 1]$.

Artificial Neuron



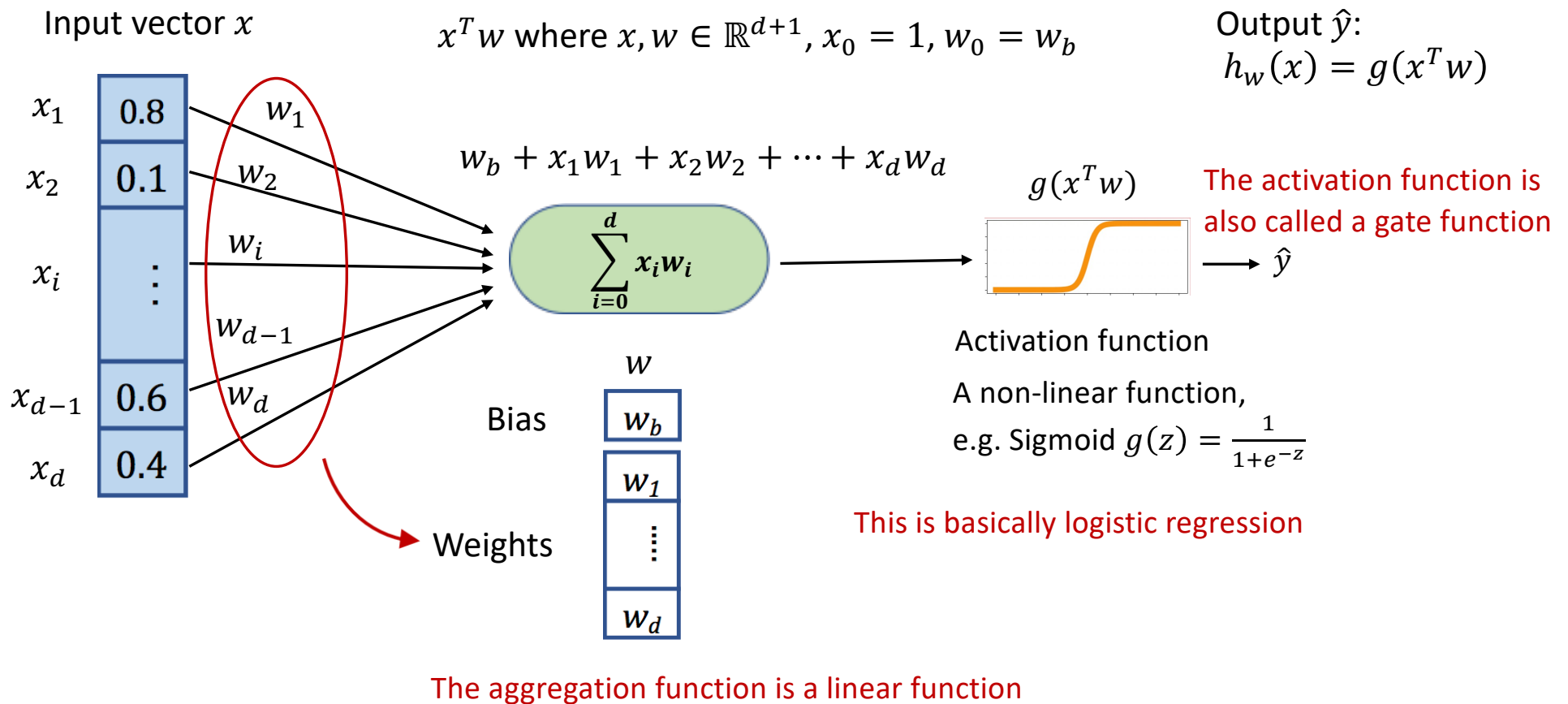
Artificial Neuron

x_0 is a dummy feature which is assigned to 1 while w_0 is the bias.

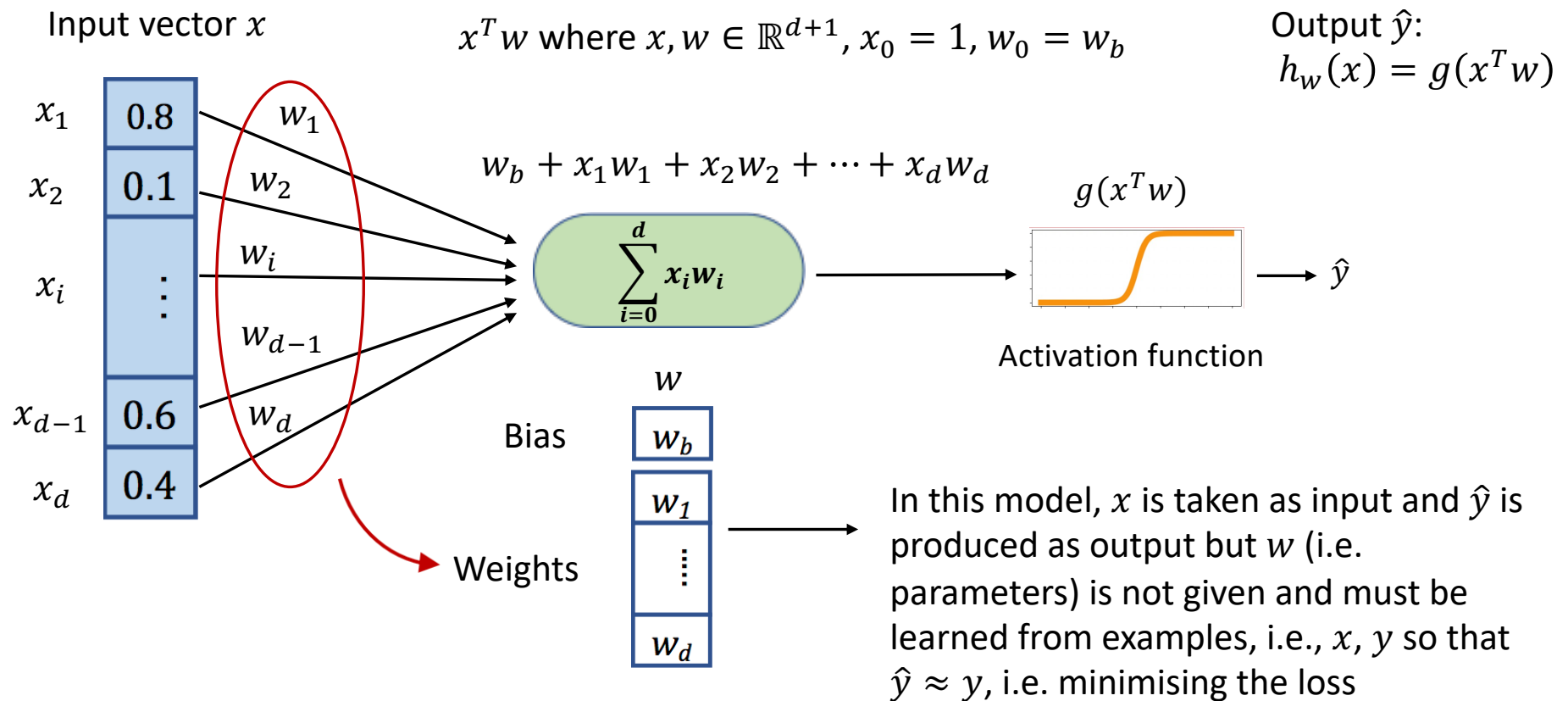


Note that we have added w_b to represent the threshold/bias for the activation function to fire. The default value of w_b is 0, i.e. there is no bias.

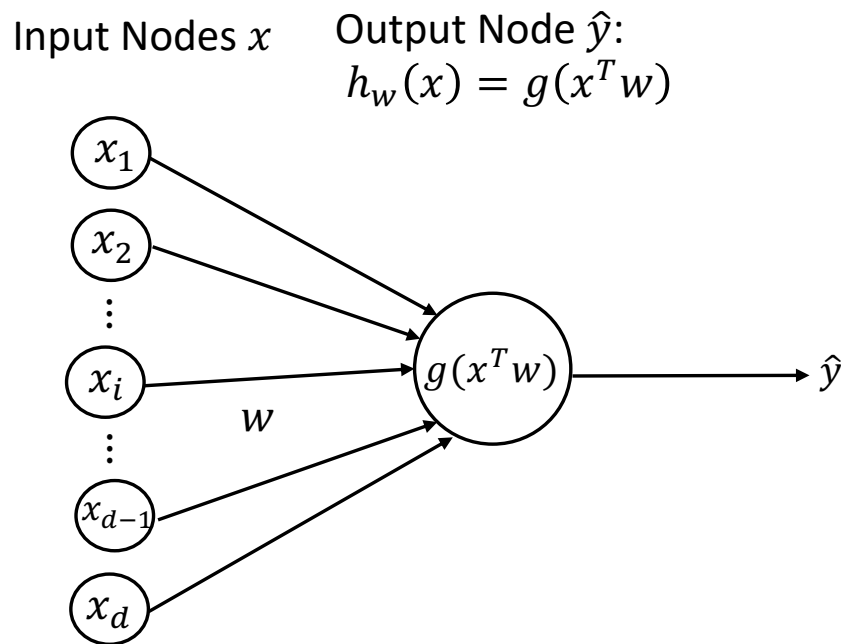
Artificial Neuron



Artificial Neuron

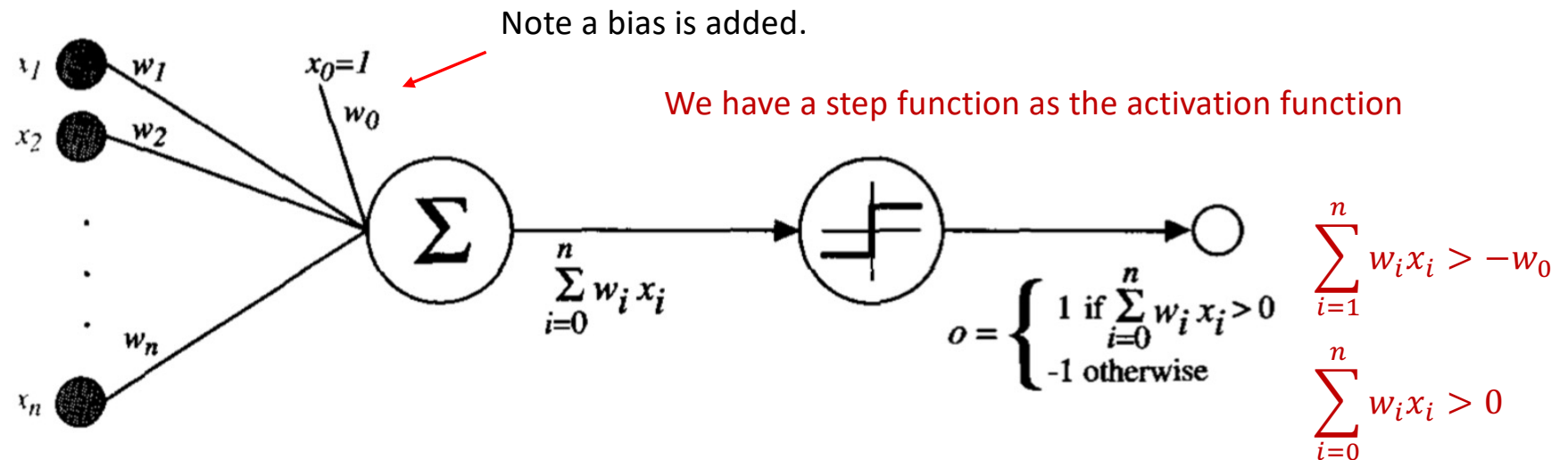


Artificial Neuron – Abstraction



We can simply combine the aggregation function and the activation function into *a node*, and represent each feature by a node too
 x is taken as input and $\hat{y}$ is produced as output but w (i.e., parameters) is not given and must be learned from examples, i.e., x, y , so that $\hat{y} \approx y$

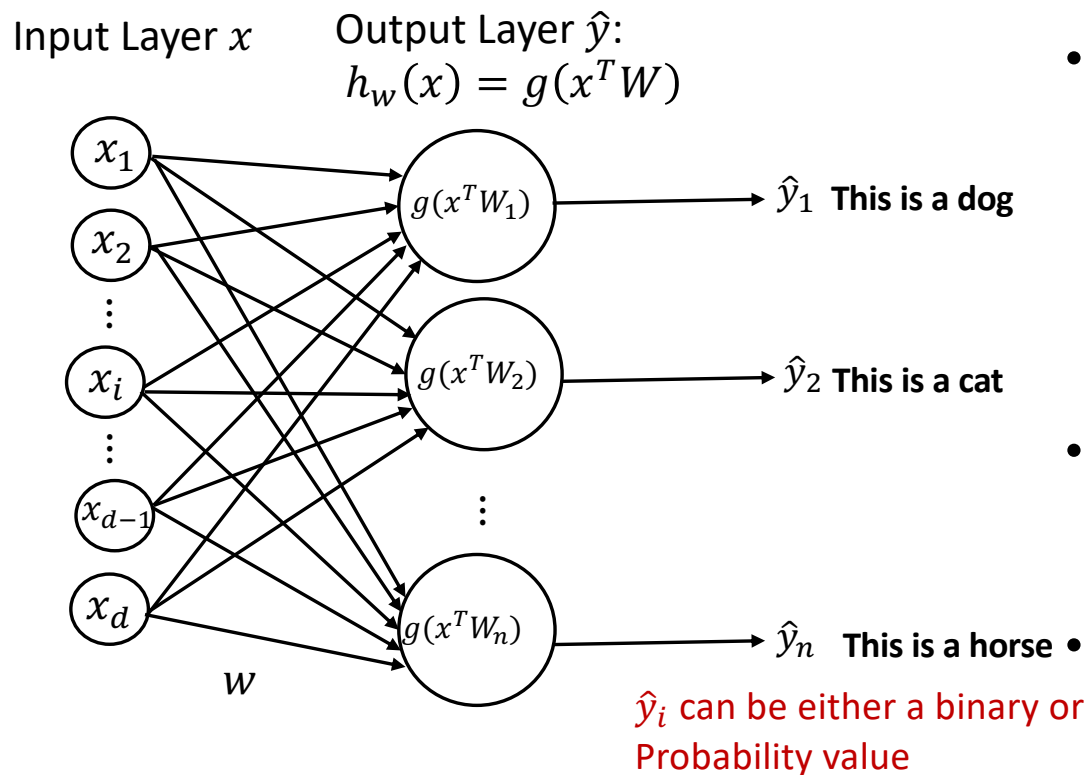
A Single Perceptron – a Special Type of Artificial Neuron



- A perceptron takes, $x_1, x_2, \dots, x_n$ with binary values (1 or 0) of as input, and produces a binary value (1 or -1) as output.
- The output is is determined by whether the weighted sum that is calculated by the aggression function is above a *threshold* ($-w_0$), which is determined by the activation function.

Artificial Neural Networks

A fully connected (dense) network of neurons – multiclass classifier

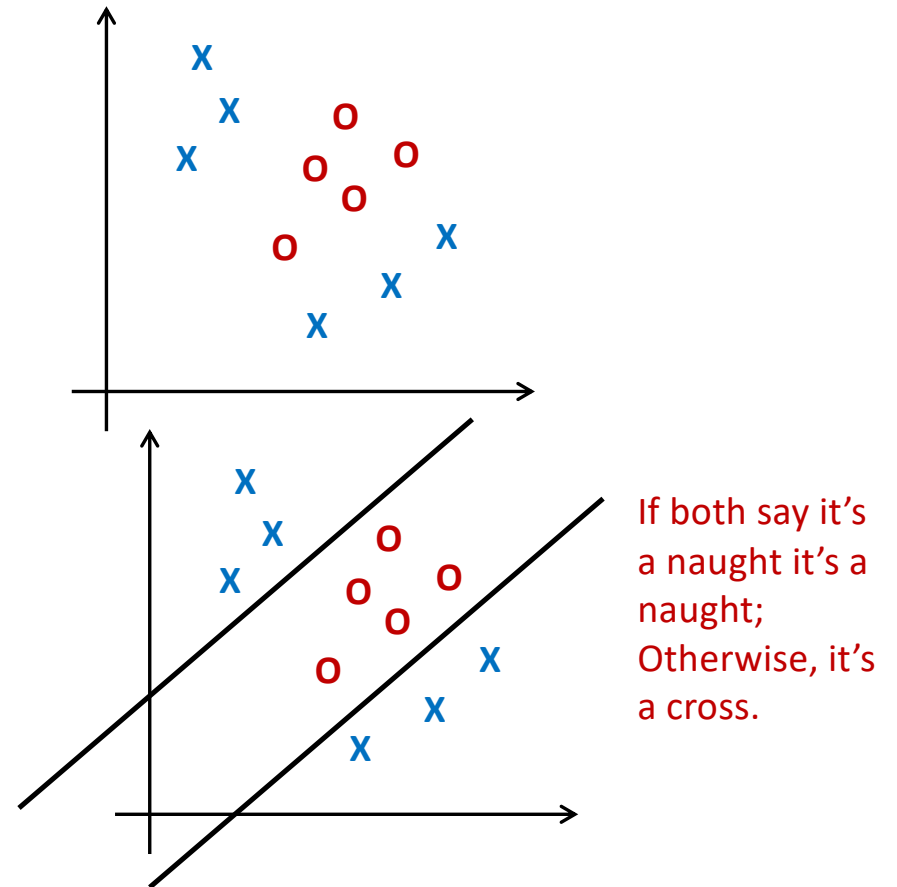


- A single-layer fully connected network (aka, single-layer perceptron *historically*), consisting of an input layer of neurons feeding forward to an output layer of neurons
- Each node takes every feature as input and produces as output a single prediction, $\hat{y}_i$
- $w^{(i)}$ is not given and must be learned from examples, i.e. x, y so that $\hat{y}_i \approx y_i$

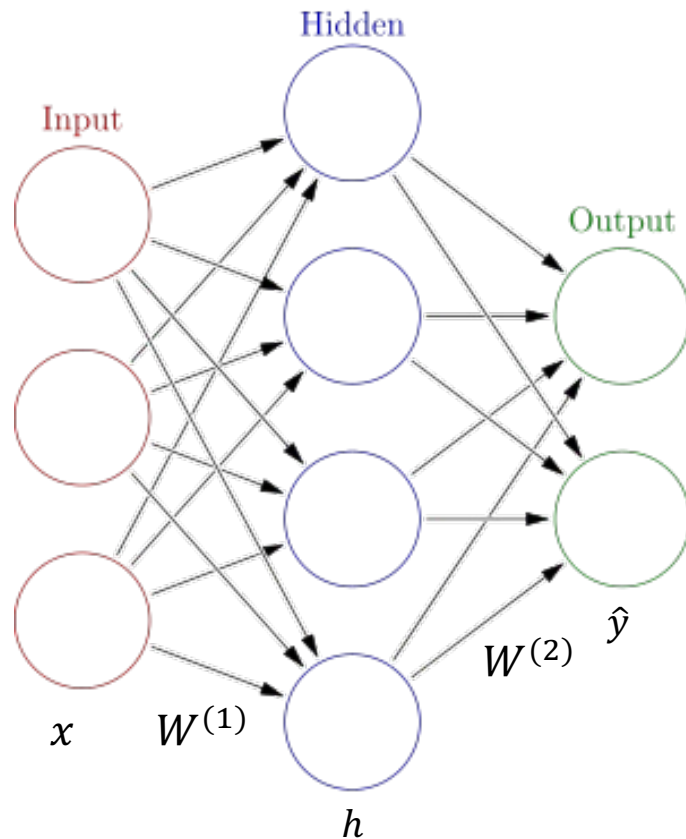
Simple/linear classifiers can be combined into complex/non-linear classifiers

The underlying idea of a single-layer fully connected network is that simple/linear classifiers can be combined into complex/non-linear classifiers layer by layer with *increasing level of non-linearity and abstraction*

For example, as shown on the right, combining two linear logistic regression classifiers represented by two straight lines would be able to do this non-linear classification

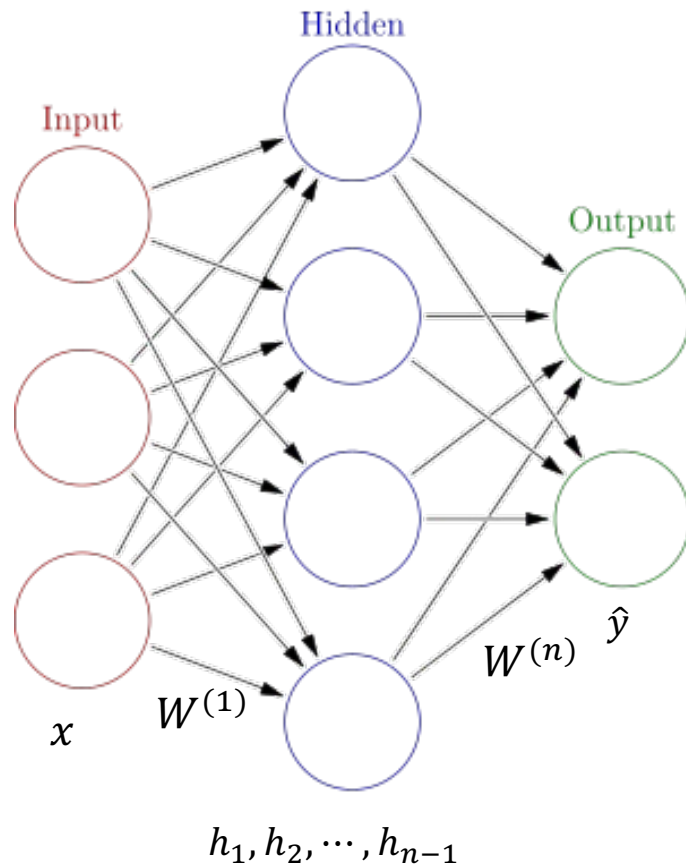


Two-layer fully connected neural networks - deep neural networks



- Two-layer perceptron - A two-layer fully connected neural network that has a hidden layer of nodes, *in addition to the input and output layers*
- Each node in the hidden layer takes every feature as input and produces as output a single prediction,
$$h = g(x^T W^{(1)})$$
- Each node in the output layer takes every output from the hidden layer and produces as output a single prediction,
$$\hat{y} = g(h^T W^{(2)})$$

Multi-layer fully connected neural networks – deep neural networks

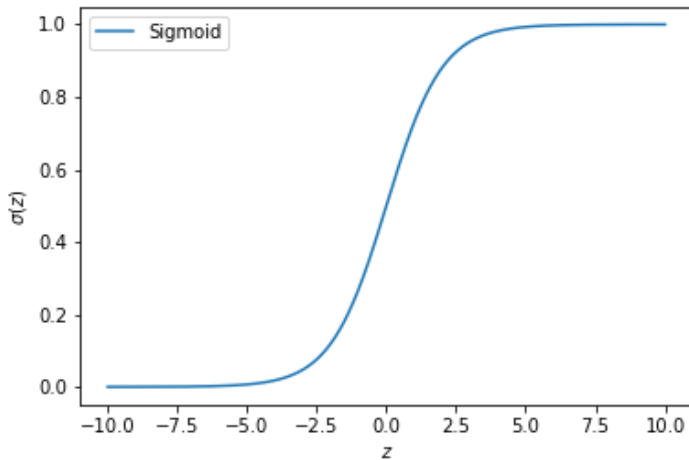


- Multi-layer perceptron - A multi-layer fully connected neural network that has $(n - 1)$ *hidden layers* ($n \geq 2$, i.e. *at least one hidden layer* in addition to the input and output layers (i.e. a generalisation of a two-layer perceptron))
- Each node in a hidden or output layer takes every output from the *previous layer* as input and produces as output a single prediction either as input to the *next layer* or as output from the whole network
- Each $W^{(i)}$, for $i = 1, 2, \dots, n$, is a *weight matrix*, with each row representing a set of weights for one node

Activation functions

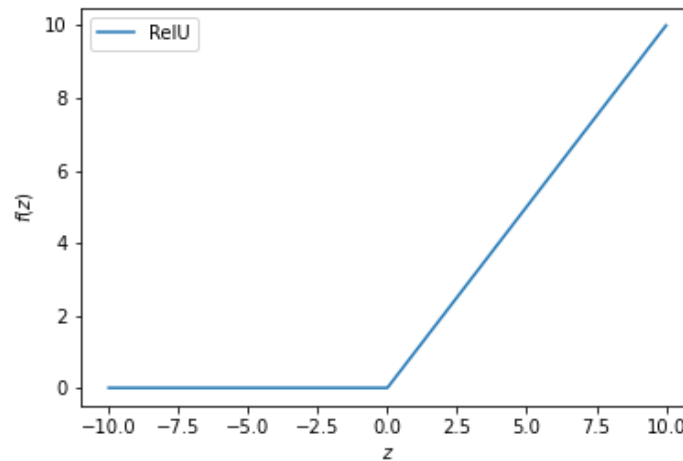
Sigmoid:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$



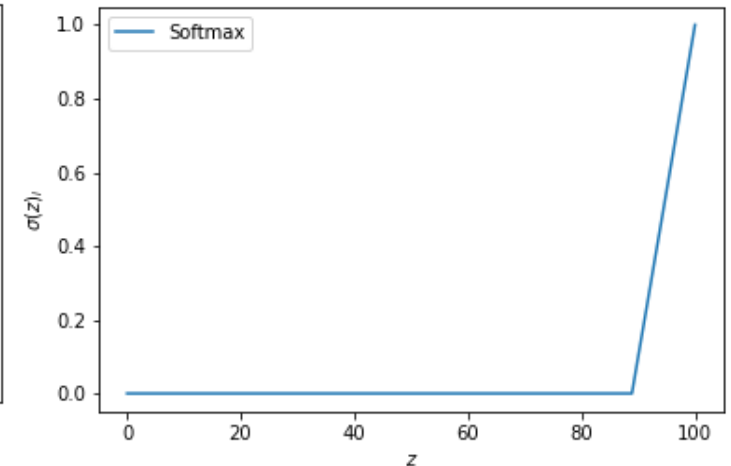
ReLU (Rectified Linear Unit)

$$f(z) = \max(0, z)$$



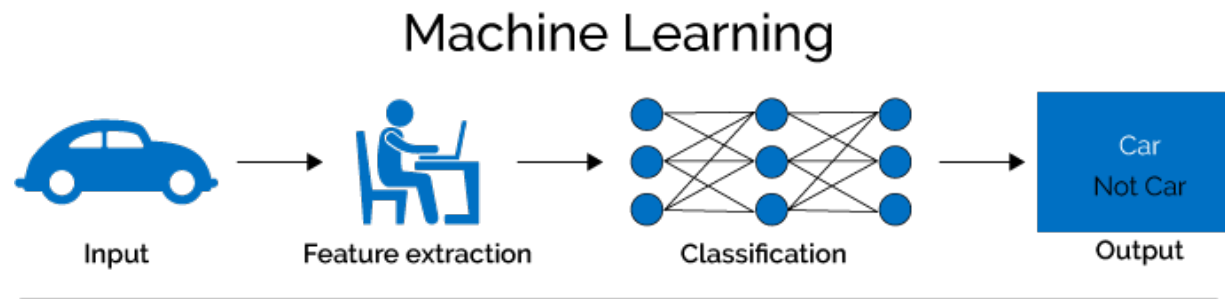
Softmax

$$\sigma(\vec{z})_i = \frac{e^{z_i}}{\sum_{j=1}^k e^{z_j}}$$

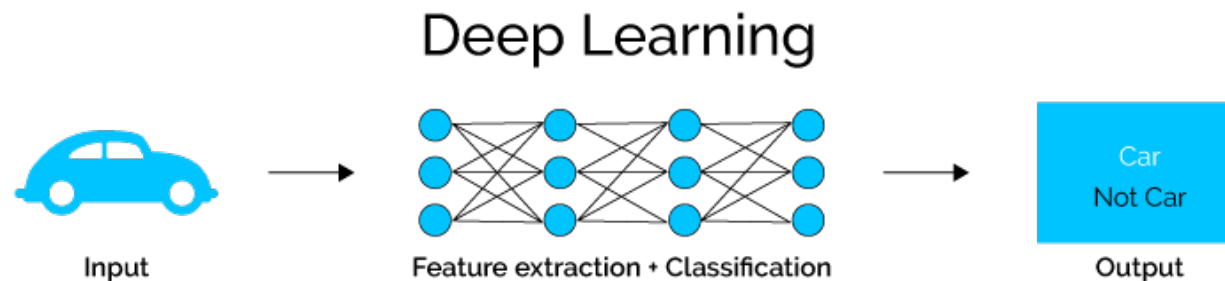


z can be replaced by an aggregation function: $\sum_{i=0}^n w_i x_i$

What is deep learning

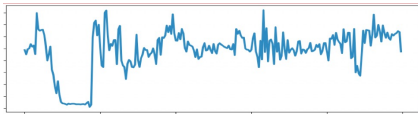


No feature extraction anymore, which is now part of a Deep Neural Network (DNN), so features become hidden too



DNN can take different types of data as input

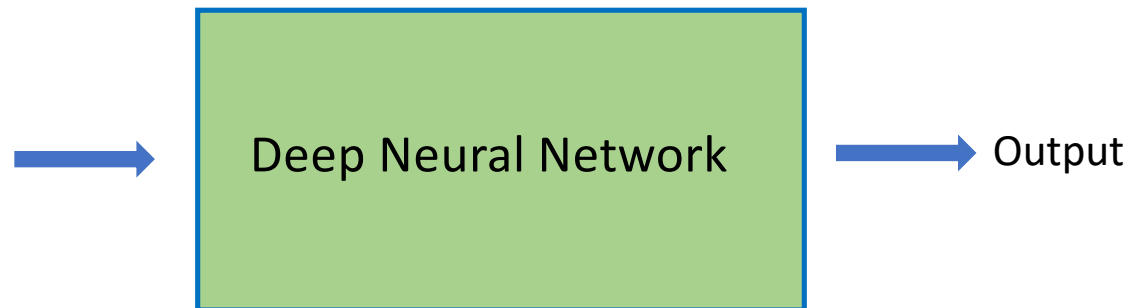
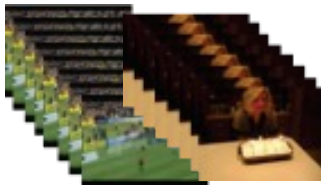
1D (text, audio, ...)



2D (image, ...)



3D (video, ...)



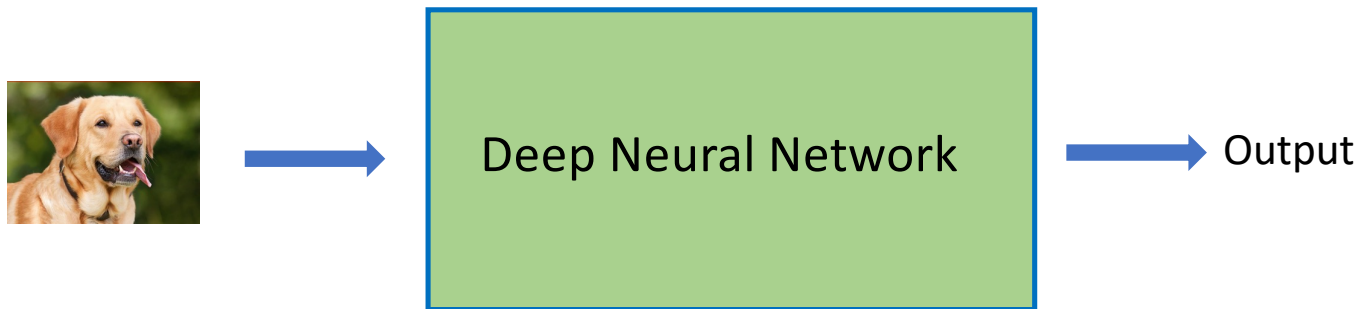
DL has many applications

- Natural language processing (NLP)
 - Audio recognition (e.g. video subtitling)
 - Speech recognition (e.g. Amazon Alexa)
 - Machine translation (e.g. Google Translate)
 - Question answering (e.g. chatbots, conversational systems)
- Computer vision
 - Image captioning
 - Image recognition
 - Action/activity recognition
- Healthcare
 - Medical image analysis
 - Disease diagnosis
 - Personalised/precision medicine
- $\vdots$

Network Training

Network Training

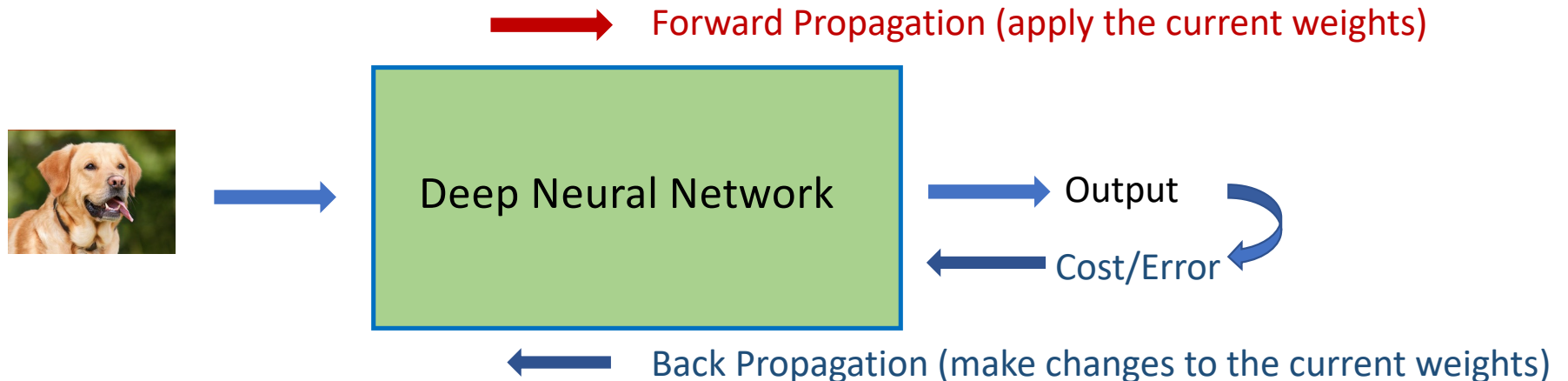
- Before the network is trained, it initially has no idea what the image is – Is it a dog, cat, horse, tiger, or ostrich, ...?
- We need to train the network, i.e., fit the parameters/weights between the hidden layers to the training dataset
- This is essentially *an optimisation problem*



What weights should we choose to produce the best output that is close the ground truth (i.e. labels)? i.e. to have the minimum cost?

Network Training - Optimisation

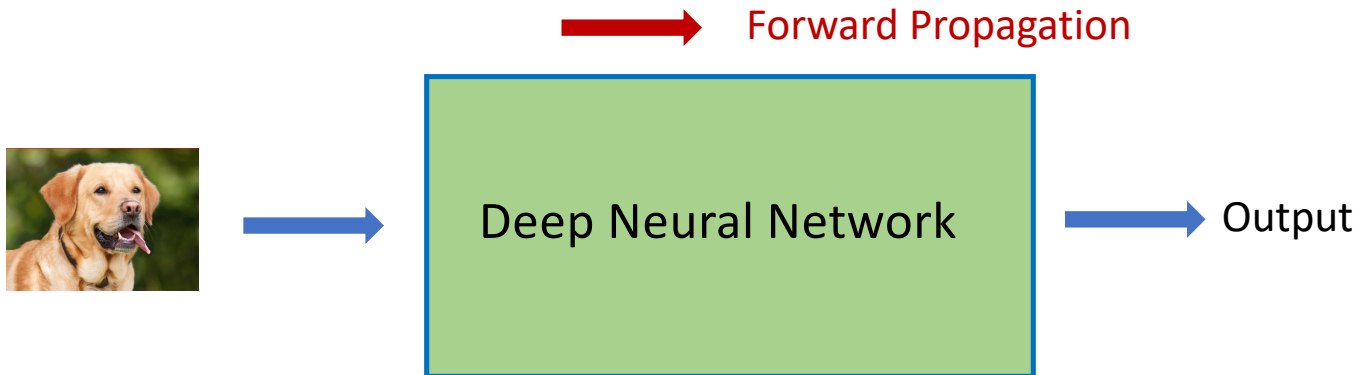
- The training algorithm produces an output (e.g., makes a prediction) using the network with the *current values of the parameters*
- It produces a validation/feedback on how good the output is, using a *cost/loss/error function*
- It updates the values of the parameters based on the feedback
- It repeats until it the trained network performs well on the test dataset



Network Training – Forward Propagation

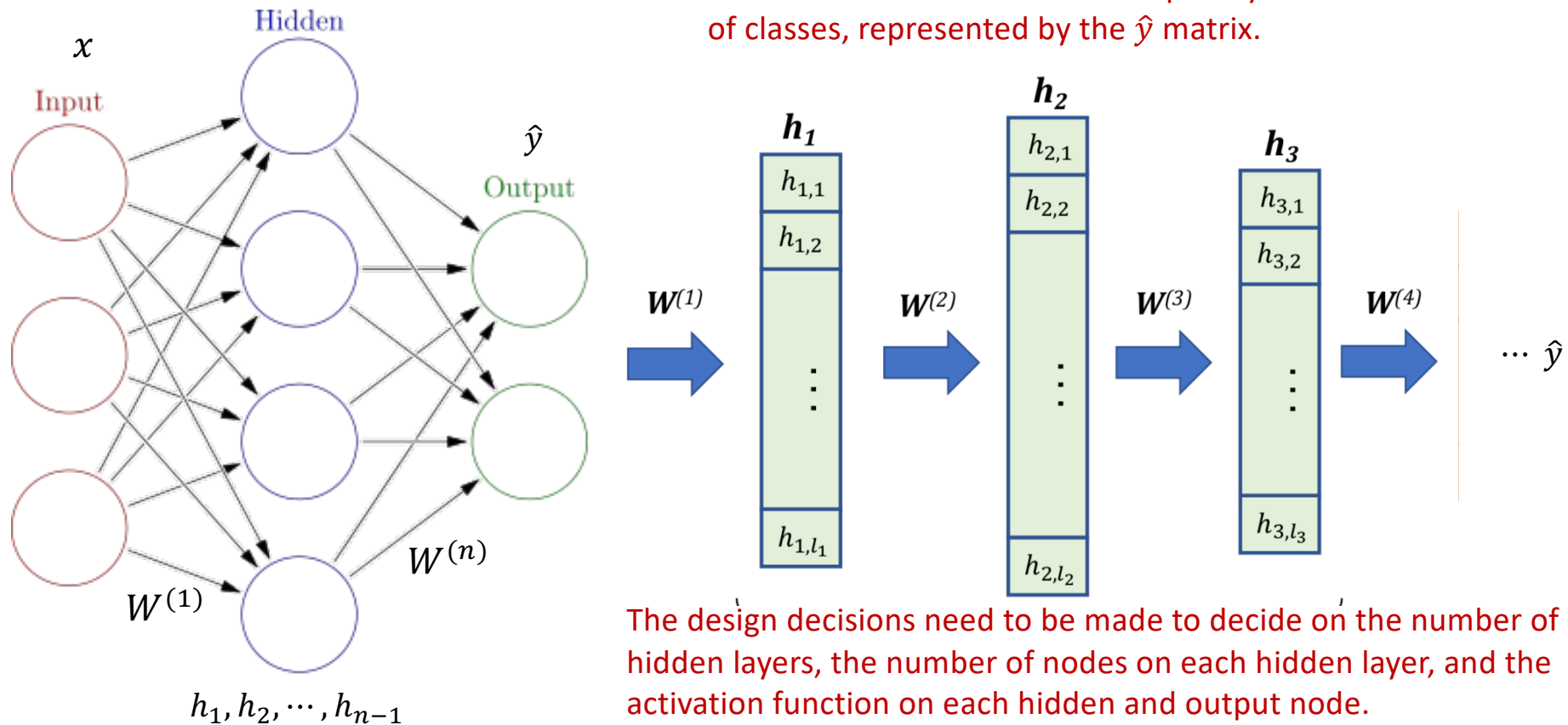
Cost function: The training algorithm needs a *cost function* to produce a validation/*feedback* on how good the output is

- It compares the output with the ground truth given by the training dataset
- It quantifies/calculates how good the output is
- It enables the training algorithm to achieve its *objective of minimising the cost function*



Forward Propagation (computes inputs to next layer/outputs)

The number of nodes on the output layer is the same as the number of classes, represented by the $\hat{y}$ matrix.

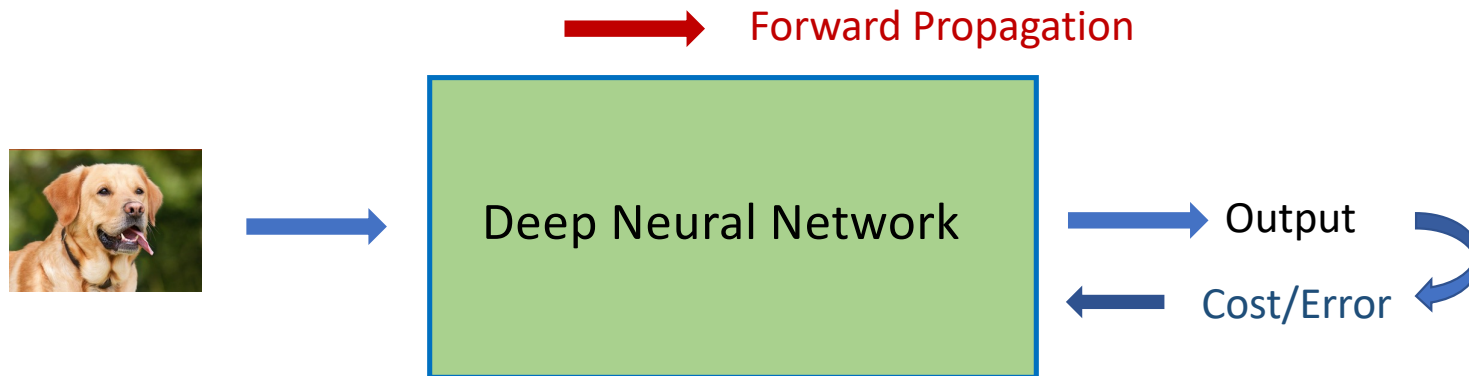


The design decisions need to be made to decide on the number of hidden layers, the number of nodes on each hidden layer, and the activation function on each hidden and output node.

The number of nodes on the input layer depends on the problem, represented by the x matrix.

Network Training – Cost Functions

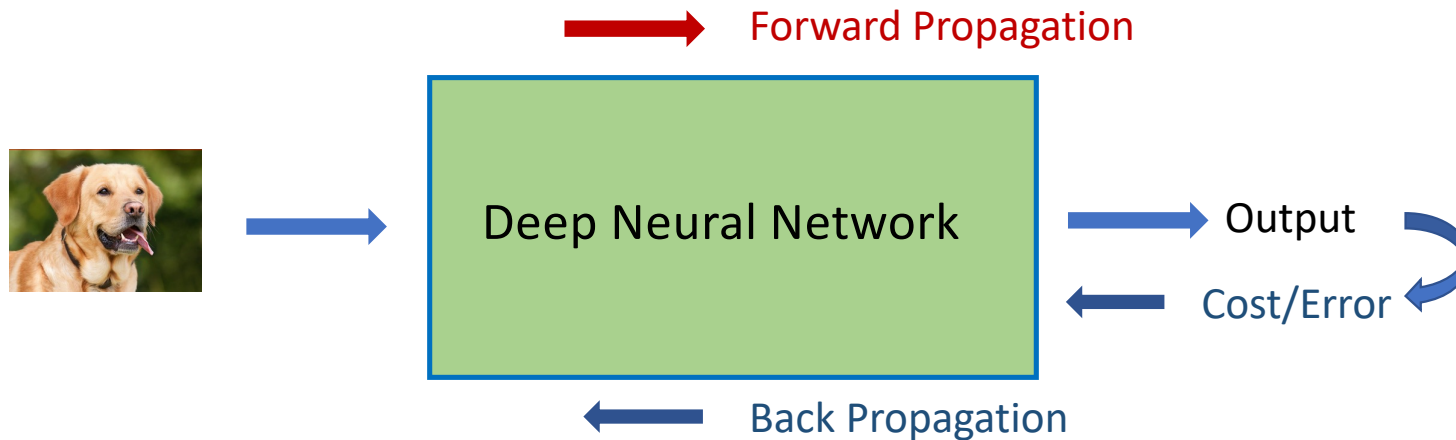
- The training algorithm needs a cost function to produce a validation/feedback on how good the output is
- Different cost functions can be define/used
 - Mean Squared Error (MSE) – for regression
 - Cross-entropy function – for classification



Network Training – Back Propagation

How to update the parameters in the network? What algorithms can be used?

- Gradient descent
- Back propagation



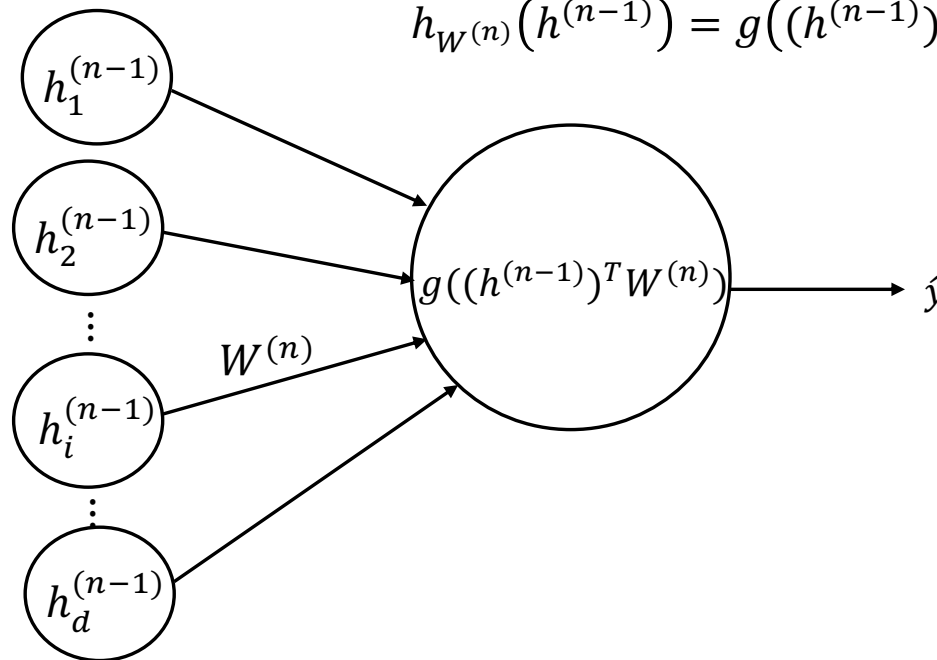
Network Training – Gradient Descent Again

- Update one set of weights by gradient descent on a layer, starting with the output layer

Input Nodes $h^{(n-1)}$

Output Node $\hat{y}$:

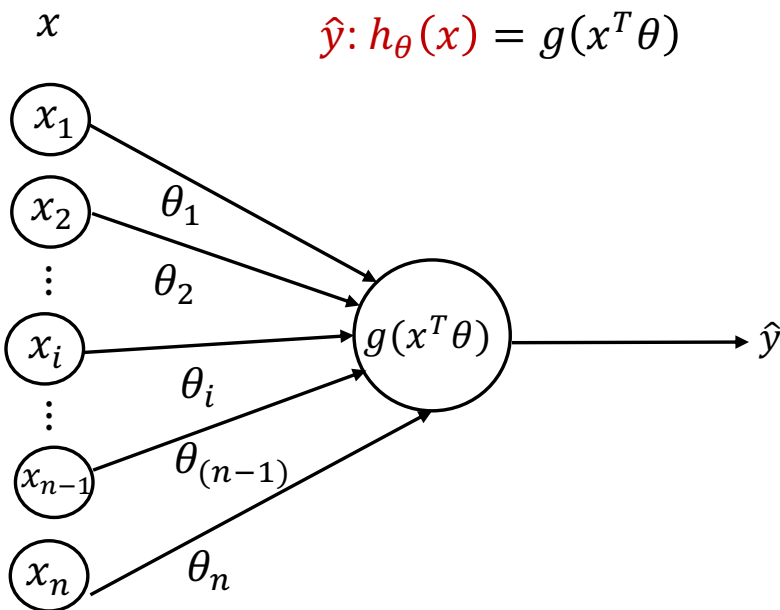
$$h_{W^{(n)}}(h^{(n-1)}) = g((h^{(n-1)})^T W^{(n)})$$



If we treat $h^{(n-1)}$ as x and $W^{(n)}$ as θ , this is just like logistic regression, as shown on next slide

Network Training – Gradient Descent Again

- Update one set of weights by gradient descent on a layer, starting with the output layer



We can use gradient descents to update θ using both the learning rate and the gradients/partial derivatives:

Step 2. Repeat changing θ to reduce $J(\theta)$ in each iteration until convergence:

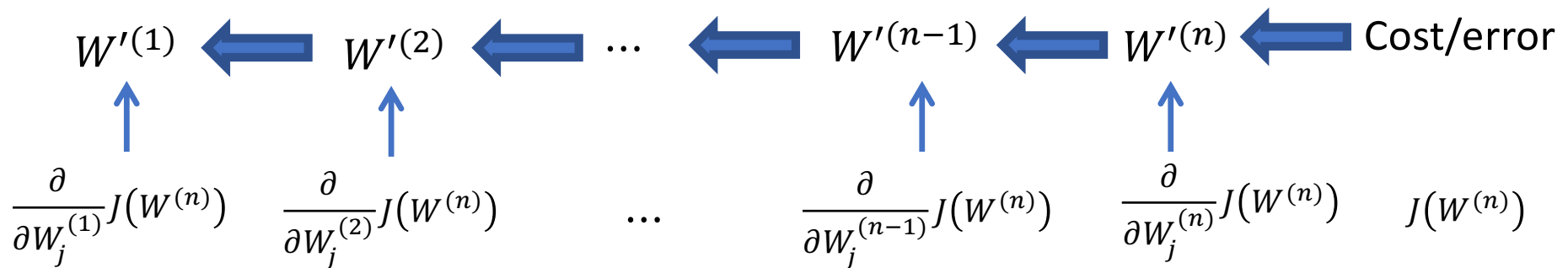
For $j = 0, 1, 2, \dots, n$ change θ_j as follows:

$$\begin{aligned}\theta_j &:= \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta) \\ &= \theta_j - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) \cdot x_j^{(i)}\end{aligned}$$

where α is called the *learning rate*, $\frac{\partial}{\partial \theta_j} J(\theta)$ represents the *gradient* of $J(\theta)$ with respect to θ_j , which is calculated as the partial derivative of $J(\theta)$ with respect to θ_j

Back Propagation (computes gradients for updating weights on previous layer)

- Update one set of weights on one layer using learning rate and the gradients/partial derivatives
- We derive the gradients/partial derivatives from one layer to the next layer backward



Each gradient/partial derivative with regard to each parameter can be computed automatically by the back propagation algorithm in the backward order, which tells how much a parameter needs to be changed towards minimising the loss function

Summary

- A multi-layer fully connected network consists of the input layer and the output layer and a number of hidden layers
- Each node on each hidden layer or the output layer consists of an aggregation function and an activation function
- Feedforward can take the input layer and pass forward the calculations through each layer to make predictions at the output layer
- The cost function can estimate how well the network is performing with the current weights
- Backpropagation can pass backward the gradients through each layer to update the weights on each layer
- This process repeats as many iterations/epochs as specified

References

- Neural Networks and Deep Learning by Michael Nielsen:
 - <http://neuralnetworksanddeeplearning.com>
 - This is a free online book
- Chapter 10 Introduction to Artificial Neural Networks with Keras, Hands-on ML.

Resources

- But what is a Neural Network? | Deep learning, chapter 1, 3Blue1Brown series S3 • E1
 - <https://www.youtube.com/watch?v=aircAruvnKk>
 - This brilliant video uses the classical digital handwriting recognition example as an introduction example, the Hello world example of deep learning (ANN).
- Gradient descent, how neural networks learn | Chapter 2, Deep learning, 3Blue1Brown series S3 • E2
- What is backpropagation really doing? | Chapter 3, Deep learning, 3Blue1Brown series S3 • E3
- This series of videos go with the book written by Michael Nielsen, Neural Networks and Deep Learning chapter by chapter.

Resources

- A Neural Network Playground
 - <http://playground.tensorflow.org/>
 - A demo from Google Tensorflow where you can choose different options, e.g. a dataset, features, number of hidden layers, number of neurons on each hidden layer, training/test split, level of noise, batch size, weights, meta-parameters, etc., to play with a neural network
 - A brilliant demo showing all the relevant concepts and how they work together in an ANN